OceanBase

本体驱动
企业级 DataAgent 的效果优化之路

吉剑南
OceanBase AI平台负责人

DACon

DataFun.

OceanBase

目录

CONTENT

今天的 AI 问数，为什么还不靠谱

RAG 的两个绕不过去的问题

问题一：相关知识，根本找不对

OceanBase

比如用户问 **“经营情况”**——真正该召回的是这 5 个指标：

向量召回 1 / 5

一般性存款余额	✕ 相似度太低，漏掉
各项贷款余额	✕ 相似度太低，漏掉
净利润金额（利润表）	⚠ 字面沾边，召回
中间业务收入	✕ 相似度太低，漏掉
贷款不良率	✕ 相似度太低，漏掉

相似度 ≠ 业务相关性。“经营情况该看哪些指标”是**业务知识**，不是字面相似——这些指标和问题八竿子打不着，向量召回按相似度抓 top-k，根本够不着 → **答案天生残缺**。

问题二：往下钻不动，答不了为什么

OceanBase

上一题答出“经营情况”的 5 个表层指标（是多少）——用户立刻追问 **“为什么？”**

一般性存款余额 （5 个表层指标之一）

⌄⌄ 它不是一个数字，是一串子指标按口径算出来的 ⌄⌄

个人活期存款余额	个人定期存款余额	个人通知存款余额	个人大额存单余额
个人特色存款余额	个人准贷记卡存款余额	个人保证金活期存款余额	个人保证金定期存款余额

要答“为什么”，得顺着这棵口径树往下钻，定位到底是哪个子项动了——这才是分析，不是查数

可 RAG 钻不下去：① 这些子指标离“经营情况”更远，**比表层指标更召不回**；②“由谁构成”是结构化的**口径关系**、不是字面相似——卡片之间**没有这条线**。于是它只能停在表层数字，答不了“为什么”。

两个问题，同一个根

OceanBase

RAG 先把企业完整的业务知识**打成碎片**，

再按相似度从碎片堆里抓取——

碎片之间，既没有**“类型”**，也没有**“关系”**

没有结构

→ 相关知识召不回（只能按相似度猜）

+

没有关系

→ 在“为什么”面前卡死

解法：把企业建成一张**有结构、能行走的图**——本体驱动

OceanBase

目录

CONTENT

本体驱动：把企业建成一张能走的图

把散落的资产连成有结构、能行走的整体

DACon

DataFun.

本体语义层：坐在数据与 AI 之间

OceanBase

经营分析　用户洞察　产品优化　成本控制　战略决策

数据源
业务系统 · 日志 · 三方 · IoT

数据处理与 ETL
清洗 · 转换 · 整合 · 治理

数据资产管理
元数据 · 标准 · 质量 · 血缘

数据存储与计算
存储 · 计算 · 调度 · 容灾

AI 应用

本体化语义层
统一定义　语义建模　指标治理　业务映射　可执行口径

数据底座
离线数仓　实时数仓　湖仓　OLAP

可视化分析
图表 · 仪表盘 · 报表 · 地图

业务应用
看板 · 报表 · 自助 · 移动

AI 应用与智能分析
问答 · 预测 · 异常 · 归因

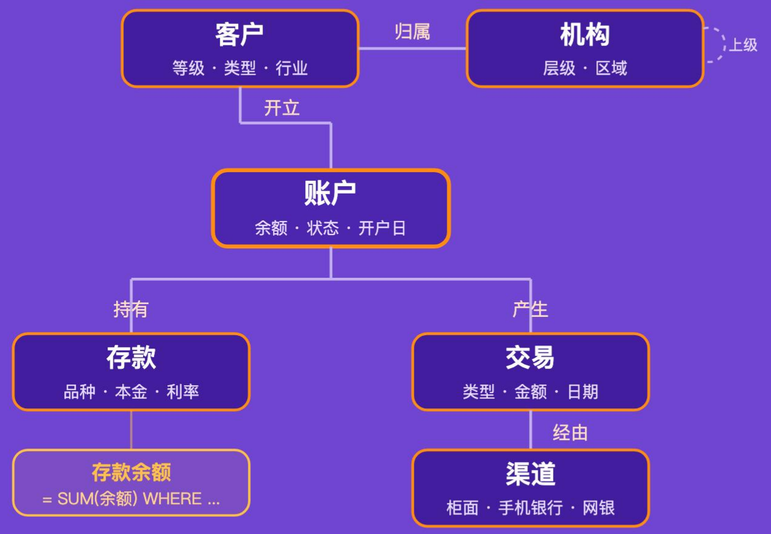
数据服务与开放
API · 集市 · 权限 · 审计

本体语义层是整套架构的中枢——连接数据与 AI 的桥梁

本体语义的四大要素

OceanBase

业务对象	业务关联	业务属性	业务口径
现实世界的数字映射	对象之间的业务关系	对象身上的业务语义	挂在对象上的计算逻辑



AI 顺着图走：归因解析

用户问题

哪些客户导致了华东区存款余额下降?

推理路径

① 客户 → ② 客户↔账户 → ③ 账户↔交易 → ④ 交易↔机构

归因结果 （影响华东区存款余额下降的客户）

客户	关系链路	贡献值	影响
恒通集团	客户→账户→交易→机构	−820万	定期到期
瑞丰实业	客户→账户→交易→机构	−650万	活期转出
沪信贸易	客户→账户→交易→机构	−470万	结构性赎回
申城物流	客户→账户→交易→机构	−380万	保证金释放

口径追溯

一般性存款余额 = 活期(2301) + 定期(2302) + 通知(2303) + ...

AI 沿着口径公式展开，定位哪个子项在拖累

四个要素如何协作：

对象告诉 AI 有哪些业务实体　关联告诉 AI 怎么走

属性让 AI 区分近义对象　口径让 AI 知道怎么算

一袋散卡片，还是一张连通图



一袋**没有线**的卡片

VS



一张**连通**的图

	RAG	本体驱动
业务对象	文本碎片，无类型区分	客户、账户、存款——每个点有身份
业务关联	碎片之间没有连接	开立、持有、归属——关联是明确的
业务口径	散在文档里，靠字面匹配	挂在对象上，可执行、可追溯

OceanBase

目录

CONTENT

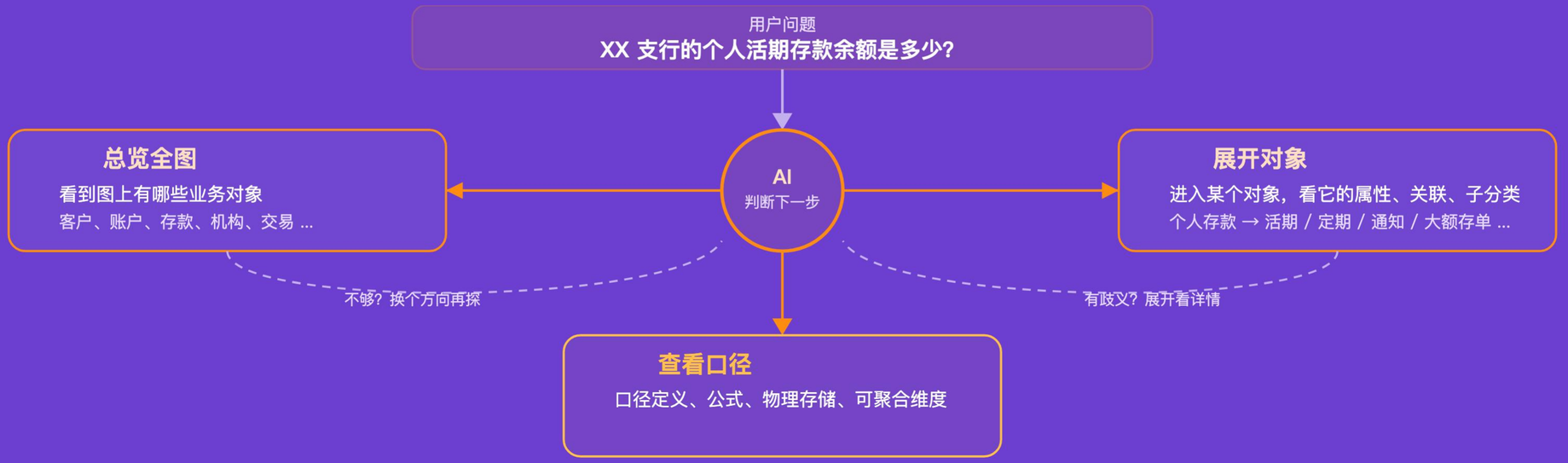
AI 怎么在本体上找答案

从一句话到一个答案的过程

DACon

DataFun.

召回策略：AI 自主探索本体图



自由导航

AI 可以随时对图中任意对象
发起探索，不受先后顺序约束

走入某个分支发现不匹配?
直接跳到另一个对象继续探

→ 没有"返回"操作，每次探索彼此独立

多路并行

AI 可以同时展开多个候选分支
综合比较后选择最匹配的方向

"存款余额"可能在个人金融下，
也可能在综合管理下——两条都探

→ 不会被锁定在某条路径上

低成本试错

一次错误的探索只是多看了几行信息
不产生副作用，不影响最终结果

错选了含保证金的口径?
看一眼公式就知道不对，换一个

→ 探错了回头的代价 ≈ 零

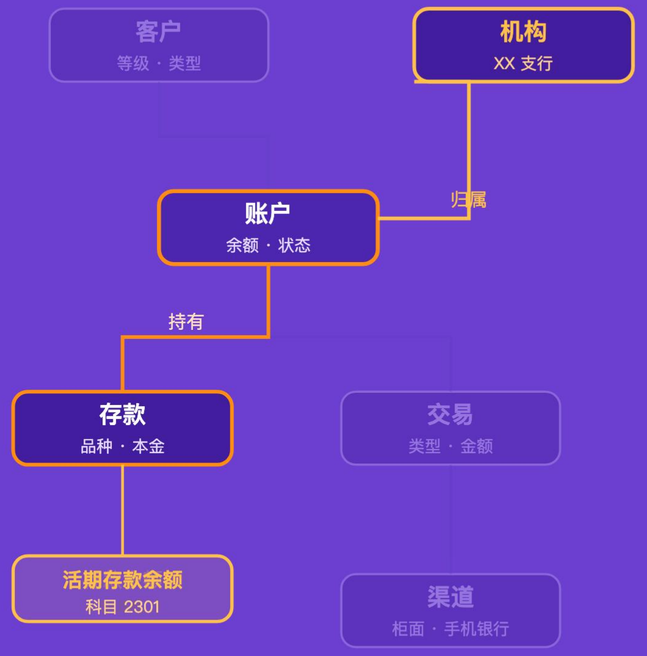
AI 手里有工具，图上有地图——**先看哪个、往哪走、什么时候够了，全是它当场判断的**

AI 在图上定位口径

OceanBase

用户问题

XX 支行的个人活期存款余额是多少?



Step 1 锚定

AI 把"个人活期存款"落到图上的"存款"对象

展开口径列表，靠业务定义定位到"个人活期存款余额（科目 2301）"
不是比名字相似——是靠口径的身份区分

Step 2 沿关联走

顺着"持有"关联往上走到"账户"，确认取数路径

账户 —持有→ 存款 —挂载→ 活期存款余额

Step 3 绑定维度

识别"XX 支行"→ 沿账户的"归属"关联找到机构对象

维度约束确定，查询条件完整

Step 4 生成查询

SELECT Diff_Balance FROM T0837... WHERE Acc_Code='2301' AND Branch='XX支行'
整条路径由 AI 在图上一步步探出来

从用户问题到生成查询——**AI 在本体图上自主走出了一条完整路径**

从指标出发，沿关联下钻

OceanBase

用户问题

华东区存款余额为什么降了？



Step 1 定位指标

找到"一般性存款余额"口径（上一页已完成）

Step 2 沿关联铺开维度

从口径出发，看它挂在哪个对象上，这个对象连着哪些其他对象

存款品种、客户等级、机构、渠道——AI 自己从图上读出来的

Step 3 数据驱动收窄

对每个维度取数看环比——定期存款 −22%、手机银行转出 68% 最显著

AI 判断这两个方向贡献最大，沿它们继续下钻

Step 4 定位根因

继续下钻：定期存款中，到期未续约占 68%，经手机银行转出

不只是"降了多少"，而是"哪儿降的、谁拖的、下一步该盯哪"

查指标是从 overview 往下找，归因是从指标往外铺开再下钻——方向相反，都在同一张图上完成

本质区别：语料，还是工具

OceanBase

RAG：提供语料

把相似文本交给 AI
AI 读到的是材料

能引用、总结、拼接
不知道下一步该查什么
给的是可阅读的信息

VS

本体：提供工具

把对象、关系、口径
变成可调用能力

能看 overview、展开对象、查看口径
能沿关联下钻，自己决定路径
给的是可执行的能力

RAG 把语料递给 AI，让它读；本体把工具交给 AI，让它做。差别不在“召回得多像”，而在 AI 能不能沿业务结构继续行动。

OceanBase

目录

CONTENT

这张图怎么长出来

从一个 Agent，到全企业的业务大脑

DACon

DataFun.

生命周期总览

OceanBase

冷启动
接入当天就能用

→

加工
自动翻译成业务图

→

生长
越用越准

→

汇聚全域
局部图连成全域图

→

反哺
新 agent 站在全公司肩膀上

不是一次性设计出来的大工程
是从真实使用里**自己长出来**的

加工：把一堆裸表，翻译成一张业务图

OceanBase

原料

表结构
表名、字段、类型、外键关联

数据采样
字段里的真实样本值

企业知识库
写下来的口径规则、业务定义

自动

合成

产出

业务对象：t_deposit_acct → “存款账户”

关系边：cust_id → 客户“开立”账户

属性语义：acct_status → 活跃/休眠/销户

可执行口径：一般性存款余额 = SUM(余额) WHERE 排除同业存款

初稿是机器画的，人只需要 review——扫一眼、改几处

生长：越用越准，把暗知识一点点逼出来

OceanBase

暗知识沉淀

- 用户说“这个存款余额要排除保证金存款” → 规则永久生效
- 用户问到图里没有的概念 → 缺口被补上
- 专家脑子里的判断，在使用中被**一点点逼出来**

和传统文档正好相反

传统指标文档

建完那天最准，之后一路过期

本体图

用得越久，越准、越全

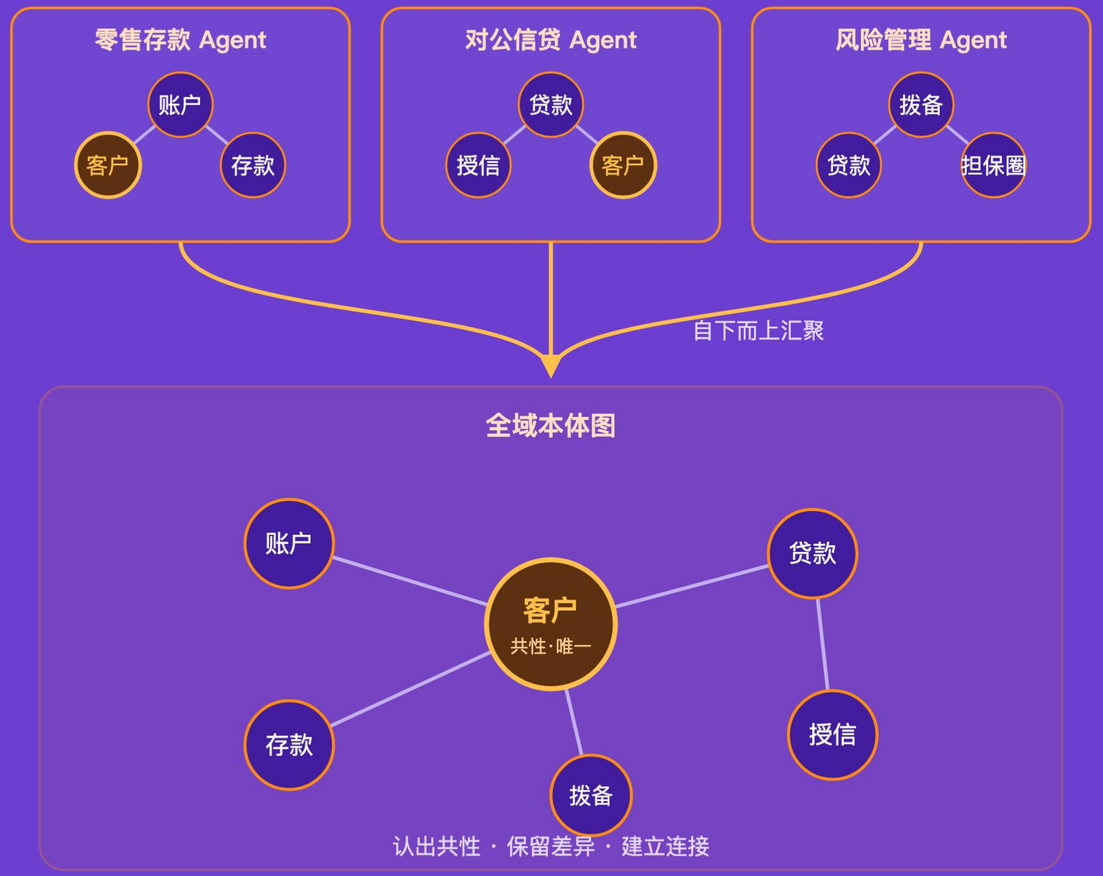
文档是建完就开始过期，这张图是用着用着越来越准。

自下而上，长成一张全域大图

从局部到全域

- 每个 Agent 在各自业务里打磨准了 → 局部精准
- 被真实业务验证过的语义，**自下而上汇聚**成全域图
- 认出共性、保留差异、建立连接——不会粗暴地并成一个

同样叫“客户”——销售关心线索和签约主体，财务关心开票和回款责任方——全域图让这些口径**都摆在那儿、可选、可追溯**



全域反哺：新 Agent 一上来就站在全公司的肩膀上

- **没有全域图**：每个新 Agent 都像新员工——概念从头学、口径重新确认
- **有了全域图**：新 Agent 直接继承全公司已经打磨准的语义，补一点专属上下文就行



进得越深，攒得越厚
攒得越厚，新场景起得越快

那为什么不一上来就铺全域?

OceanBase

直接自顶向下铺全域，几乎必踩四个陷阱：

抽象化

为了覆盖所有部门，定义只能写得特别宽泛——“客户是与企业发生商业关系的主体”。正确但没用

会议化

全域要跨部门对齐 → 一轮轮拉通会。文档上统一了，业务里还是各看各的

静态化

当成一次性工程建完就开始过期。没有高频使用牵引，很快和现实脱节

过早平台化

摊子铺得很大，业务方配合了半天，却迟迟看不到自己的问题被解决

全域是目标，不是起点。 正确的路是自下而上。

真正的壁垒，是企业自己的业务理解

OceanBase

模型会越来越强，当人人都能接上最强的模型

真正拉开差距的，是**谁能让模型更深、更准、更持续地懂自己的业务**

🔒 这张越用越厚的本体图

装的是企业自己最值钱、又最难被复制的东西：

被持续校准的业务理解

OceanBase

从找相似，到会行动

RAG 让 AI 更会读材料；

本体让 AI 开始会做业务动作。

DataPilot · 把业务理解沉淀成可执行的 AI 能力

DACon

DataFun.

OceanBase

THANKS

DACon

DataFun.